MPTA-Repair: Multi-Property Clock-Bound Repair for Industrial Timed Automata

***Abstract*—Timed automata are widely used to model and verify real-time systems with strict timing requirements. Industrial models are often checked against multiple predefined properties, including timed safety properties. When these properties are violated, the faults may stem from erroneous clock bounds in guards or invariants; however, repairing such bounds is challenging because a local change may affect other properties or alter intended untimed behavior. This paper presents MPTA-Repair, a counterexample-guided iterative approach for multi-property clock-bound repair of timed automata. It extracts repairable bounds from guards and invariants, analyzes diagnostic counterexamples, and uses MaxSMT-based repair generation to synthesize small candidate edits. Each candidate is validated against the full property set and checked for admissibility to ensure functional equivalence. Experiments on UPPAAL and XtaBenchmarkSuite benchmarks show a 93% repair success rate, while an industrial case-study dataset achieves 76%.**

***Index Terms*—Timed automata, multi-property repair, clock-bound repair, counterexample-guided repair**

I. Introduction

Real-time and industrial software is increasingly deployed in settings where timing faults may directly compromise safety, availability, or mission success. In railway systems, timed automata have been used to model train-gate controllers, communication protocols, and railway safety layers [1]. In automotive and autonomous-driving systems, they have been used to reason about gear-control logic, road-junction behavior, and time-bounded decision rules [2]. In medical cyber-physical systems, timed models have been used to analyze pacemakers and ventilator controllers, where physiological timing windows and actuation deadlines are safety-critical [3]. In aerospace and avionics, timed-automata models have supported schedulability and deadline analysis for real-time software and distributed industrial architectures [4]. Across these domains, correctness depends not only on the order of discrete events, but also on when these events occur. A system component that eventually raises an alarm, applies a pulse, sends a message, or releases a resource may still be incorrect if the action occurs too early, too late, or for the wrong duration.

Timed automata provide a well-established modeling formalism for this class of systems [5]. Real-time systems are often represented as networks of timed automata, where clocks, guards, invariants, synchronizations, and resets describe both discrete behavior and quantitative timing constraints [6]. Verification is then performed against temporal properties that may include safety, bounded response, deadline, error-state reachability, alarm-duration, and deadlock-freedom requirements. When a property is violated, a real-time model checker such as UPPAAL, Kronos, or opaal can produce a timed diagnostic trace (TDT), i.e., a timed counterexample that shows how the model reaches a violating state [7].

However, diagnosis is only the first step. Interpreting a TDT and turning it into a correct model edit remains a demanding engineering task. Realistic industrial models often contain many locations, transitions, clocks, guards, invariants, synchronizations, and resets. A TDT exposes one feasible execution that violates a property, but it does not directly identify which clock constraint is wrong, whether the defect lies in a guard or an invariant, or how a bound should be changed to restore the intended timing behavior [8]. This ambiguity becomes more pronounced when the model is checked against multiple properties. Industrial specifications are usually not single assertions, but collections of timing requirements that jointly describe safety, response, scheduling, and reliability obligations [9]. A local edit that eliminates one diagnostic trace may still leave related traces feasible, or may even make another property false. Thus, automated repair for practical industrial models must go beyond explaining a single counterexample and must validate each candidate repair against the complete property set.

Existing timed-automata repair techniques provide an important foundation. Clock-bound repair for timed systems encodes a TDT into linear real arithmetic, introduces variation variables for clock bounds, and uses MaxSMT solving to compute small modifications to guards and invariants [8]. TarTar extends this idea into an automatic repair analysis tool that suggests syntactic repairs and checks whether the repaired model preserves the untimed functional behavior of the original network timed automaton [10]. Other related directions include parameter synthesis for parametric timed automata [11], clock-guard repair through abstraction and testing [12], robustness analysis for uncertain timing constants [13], and SAT/SMT-based model or program repair [14]. These methods show that automated repair of timed models is feasible, but most existing TDT-based repair workflows remain centered on one violated property or one diagnostic trace at a time.

The engineering challenge addressed in this paper is different but complementary to existing single-trace repair and synthesis techniques [12]. A straightforward repair loop can process one violated property, collect one TDT, compute a local repair, instantiate a candidate model, and then run regression verification. This loop is useful as a baseline, but it may become inefficient or unstable when one timing fault affects several properties or several related traces. Similar traces may repeatedly encode overlapping parts of the same faulty behavior; a candidate that eliminates one trace may

still leave other violations feasible; and a bound changed for one property may affect another property when the same clock, transition, location, or synchronization participates in both executions. If all editable bounds are considered equally plausible repair targets, the solver may explore many weakly related candidates, including edits that are syntactically valid but distant from the timing decisions exposed by the observed violations. Such repairs may pass a local check while giving engineers little insight into the shared cause of the failures.

This paper presents **MPTA-Repair**, an automatic repair workflow for multi-property, multi-counterexample clock-bound repair of industrial timed-automata models. Given a faulty model, a set of timing properties, and diagnostic traces produced by model checking, MPTA-Repair suggests small modifications to numerical clock bounds in transition guards and location invariants. The repair space is intentionally restricted to clock-bound constants, rather than arbitrary structural edits, specification changes, synchronization changes, or reset changes. The workflow uses lightweight relation analysis among properties, diagnostic traces, and editable clock bounds to guide repair search, but this analysis is only an optimization strategy. Every accepted candidate must satisfy the complete property set through regression verification and must pass a TARTAR-style admissibility check based on functional equivalence. In this way, MPTA-Repair aims to generate repairs that are not only local counterexample fixes, but also system-level timing corrections validated against the full set of industrial timing requirements.

We evaluate MPTA-Repair on two dataset groups. The benchmark dataset is collected from UPPAAL and XTA benchmark sources and is used to exercise the repair workflow on established timed-automata examples. The industry dataset is collected from practical industrial case studies, including railway alarm and communication scenarios [1], [15], train-gate behavior [16], automotive gear control [2], cardiac pacing [3], autonomous road-junction rules [17], mechanical ventilation [18], and aerospace scheduling [4]. The evaluation measures repair effectiveness, repair size, runtime, regression results, and behavior preservation. We compare MPTA-Repair with an iterative TARTAR-style baseline under the same full-property regression and admissibility requirements, and report practical observations from applying automated repair to industrial timed-automata models.

This paper makes the following contributions:

1) We formulate clock-bound repair for practical industrial timed-automata models as a multi-property and multi-counterexample repair problem.
2) We present MPTA-Repair, an automatic repair workflow that uses diagnostic traces and lightweight relation analysis to guide clock-bound repair.
3) We implement a prototype repair pipeline combining diagnostic trace analysis, MaxSMT-based clock-bound repair, full-property regression verification, and TARTAR-style admissibility checking.
4) We evaluate the approach on benchmark and industry datasets and compare it with an iterative single-property repair baseline under the same acceptance criteria.
5) We report practical lessons on applying automated timed-automata repair to industrial models, especially regarding property selection, repair interpretability, and the need for full regression after each candidate repair.

The remainder of this paper is organized as follows. Section II provides the background and problem setting, including timed automata, diagnostic traces, clock-bound repair, and the multi-property repair objective. Section III presents MPTA-Repair and its relation-guided repair workflow. Section IV describes the prototype implementation and evaluation setup. Section V reports the evaluation results and practical lessons learned from the benchmark and industry datasets. Section VI discusses threats to validity and related work. Section VII concludes the paper.

II. Background and Motivation

A. Timed Automata and Networks

We use timed automata as the formal model for real-time industrial behavior [5]. Let C be a finite set of clocks. A clock constraint over C is a conjunction of atomic constraints of the form $c \sim n$, where $c \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and n is a non-negative numerical bound [8]. We write $B(C)$ for the set of such clock constraints. A timed automaton is a tuple $T = (L, l0, C, \Sigma, \Theta, I)$, where L is a finite set of locations, $l0 \in L$ is the initial location, Σ is a set of action labels, Θ is a finite set of transitions, and $I : L \to B(C)$ assigns an invariant to each location [6]. A transition $\theta \in \Theta$ can be written as (l, g, a, r, l'), where l and l' are the source and target locations, $g \in B(C)$ is the guard, $a \in \Sigma$ is the action label, and $r \subseteq C$ is the set of clocks reset by the transition.

The semantics is defined over states (l, u), where l is a location and $u : C \to \mathbb{R}_{\geq 0}$ is a clock valuation. A delay transition lets time elapse in the current location while its invariant remains true. An action transition can be taken when its guard is satisfied; it moves the automaton to a target location and resets the specified clocks. Thus, guards constrain when discrete actions are enabled, while invariants constrain how long the automaton can remain in a location. Networks of timed automata compose several such automata, often representing different parts of an industrial system, such as a controller, plant, environment, communication protocol, monitor, or scheduler [19]. We use this industrial framing deliberately: protocols, schedulers, embedded controllers, and monitored physical processes may all appear in practical industrial timed-automata models.

Timed automata are particularly suitable for practical industrial models because many requirements are expressed as numerical timing thresholds. For instance, vehicle controllers may need to complete a gear-shifting sequence within a bounded response time [2]; railway controllers may need to satisfy gate, alarm, or communication timeout requirements [1]; medical devices may encode physiological timing windows and refractory intervals [3]; and avionics or aerospace

scheduling models may encode execution-time bounds and deadlines [20]. In these models, a numerical bound in a guard or invariant often represents a design threshold, timeout value, sampling period, alarm duration, or resource deadline.

This paper focuses on faults in such numerical clock bounds. The assumption is not that model structure is irrelevant, but that in many engineering settings the designer has already captured the intended discrete structure while some timing constants may be inaccurate due to calibration, dimensioning, transcription, or design-space uncertainty. This fault model is consistent with the motivation of clock-bound repair: a clock-bound change can correct a clerical modeling mistake or indicate that the timing resources of the design need to be re-dimensioned [8]. It is also consistent with the broader intuition behind mutation-based repair and testing that many defects can be modeled as small deviations from an otherwise nearly correct artifact [21].

B. Timed Diagnostic Traces

Timed-automata models are checked against timing properties using real-time model checkers [19]. In this paper, we focus on timed safety properties, i.e., properties that require all reachable states to satisfy a specified invariant condition. Such properties commonly constrain whether a location may be reached, whether a clock may exceed a bound in a location, or whether an error state is reachable. More complex requirements can also be represented using observer automata or monitor templates that reduce them to safety-style checks [9].

A symbolic timed trace is a finite sequence of transitions $S = \theta 0, \ldots, \theta n-1$ [8]. A realization of S is a sequence of non-negative delays that makes the transition sequence executable from the initial state while respecting guards, resets, and invariants. A trace is feasible if it has at least one realization. If a feasible trace has a realization whose final state violates a timed safety property Π, then the trace is a timed diagnostic trace for Π [8].

A TDT is valuable because it gives concrete evidence of a timing violation. It records a sequence of timed and discrete behavior leading to a property-violating state, depending on the trace representation produced by the model checker. However, it is not a root-cause explanation [8]. A TDT does not state which numerical bound is wrong, whether the relevant fault is in a guard or invariant, or how the model should be modified. Multiple clock constraints can jointly enable the violating realization, and one incorrect bound can contribute to more than one TDT. Therefore, TDTs are best understood as repair evidence rather than complete repair instructions.

This distinction is important in industrial models. A repair that simply blocks the discrete path of a TDT may remove the observed violation but also remove an intended functional behavior, such as a system action, communication response, scheduling transition, or medical actuation. A valid repair should instead correct the timing behavior while preserving the relevant untimed functionality of the model [8].

C. Clock-Bound Repair

Clock-bound repair modifies numerical bounds in clock constraints while preserving the surrounding automaton structure [8]. In this paper, a repairable bound is an occurrence of a numerical constant n in an atomic clock constraint $c \sim n$ that appears in a transition guard or a location invariant. A repair changes n to a new value n', or equivalently applies a variation v so that the repaired bound becomes $n + v$.

The current MPTA-Repair pipeline does not modify property formulas, add or delete locations, add or delete transitions, change synchronization labels, replace clock references, or add/remove clock resets [12]. This scope is deliberate. We aim to repair timing-parameter faults in a model whose discrete behavior structure is treated as the intended design. This avoids repairs that satisfy a property by changing the specification, altering the modeled protocol structure, or disabling intended behavior [8]. It also keeps the repair output understandable to engineers, because each edit corresponds to a concrete timing threshold in the model.

TDT-based clock-bound repair encodes the feasibility of a diagnostic trace as constraints over delays and clock values [8]. Bound variation variables are then introduced for clock constraints occurring in guards and invariants. MaxSMT solving is used to find a small set of bound changes that prevents the trace from having a realization that violates the property [22]. In the original single-trace setting, this yields a local clock-bound repair: the same discrete trace remains executable, but no timing realization of that trace witnesses the considered property violation.

A local repair is not automatically valid for the full model. Changing clock bounds can affect other executions, other properties, and the reachability of functional behavior. The TARTAR approach therefore defines admissibility using functional equivalence: the repaired model should preserve the untimed language of the original model, so that no action traces are added or removed by the repair [8]. MPTA-Repair adopts this admissibility criterion as the behavior-preservation gate for accepted candidates.

D. Multi-Property Repair Objective

Industrial timed-automata models are usually validated by a set of timing properties rather than by a single assertion [9]. These properties jointly express the expected safety and reliability envelope of the model, such as bounded response, absence of unsafe states, timeout handling, deadline satisfaction, and deadlock freedom [23]. Let M_bad be a faulty industrial timed-automata model, and let $\Phi = \{\varphi 1, \ldots, \varphi m\}$ be the property set used to validate it. The model may violate one or more properties, and each violated property may produce one or more TDTs. The goal is to construct a repaired model M_rep by modifying a small number of clock-bound constants in guards and invariants.

A candidate repair is accepted only if it satisfies three requirements. First, the repaired model must remain syntactically valid as a timed-automata model; in particular, repaired clock

bounds must remain non-negative, and rational bounds introduced by solving must be normalized if required by the target tool representation. Second, the repaired model must satisfy the complete property set Φ under regression verification, not only the property that produced the current TDT. Third, the repaired model must preserve the functional behavior of the original model using the TARTAR-style admissibility test based on functional equivalence [8].

The multi-property setting imposes obligations beyond local single-trace repair [24]. Different properties may share clocks, locations, synchronization actions, or execution fragments. A repair computed for one property can therefore affect the behavior relevant to another property. Similarly, multiple TDTs may reflect the same underlying timing defect, so repairing them independently may duplicate solving effort or produce inconsistent candidate edits. MPTA-Repair does not treat these properties and traces as unrelated subproblems. Instead, it analyzes them as a joint repair task, generates candidates using multi-property and multi-counterexample evidence, and accepts a repair only after iterative regression verification and admissibility checking.

III. Approach

A. Overview

MPTA-Repair is a counterexample-guided repair workflow for multi-property, multi-counterexample clock-bound repair of industrial timed-automata models [25]. Its input is a faulty model M, a property set $\Phi = \{\varphi 1, \ldots, \varphi m\}$, and configuration parameters such as the maximum number of refinement rounds, candidate limits, and a global timeout. The repairable bounds are not given as a separate user input; they are extracted from the numerical clock bounds occurring in transition guards and location invariants [8]. The output is either a repaired model M′ together with a set of clock-bound edits, or a failure report explaining why no admissible full-property repair was found within the configured budget.

The workflow is designed around two principles. First, candidate repair generation may be driven by local diagnostic traces, but candidate acceptance must be global. A candidate that repairs one violated property may cause another property to fail, because the same clock, location, transition, or synchronization may participate in several timing requirements. MPTA-Repair therefore keeps collecting counterexamples from newly violated properties and uses them to further restrict the set of admissible repair candidates. This refinement continues until a candidate satisfies the complete property set and passes admissibility checking, or until the configured budget is exhausted. In our evaluation, we use a fixed timeout, for example 10 minutes per repair instance, to avoid unbounded refinement [26].

Second, multiple properties and multiple diagnostic traces are not treated as unrelated repair subproblems. A single faulty timing bound may contribute to several violations, and several traces may expose different timing realizations of the same underlying defect. MPTA-Repair therefore maintains a pool of property-trace obligations and searches for a clock-bound assignment that satisfies all currently encoded obligations. Relation analysis among properties, traces, and repairable bounds is used to reduce redundant work and guide the search, but it is not used as a substitute for verification [25].

At a high level, MPTA-Repair proceeds as follows. It verifies the model against all properties, collects TDTs for violated properties, extracts repairable clock-bound values, and builds a joint MaxSMT repair problem from the current property-trace pool. The resulting assignment is instantiated into a candidate model. The candidate is then checked against the complete property set and against a TARTAR-style admissibility criterion [8]. If validation fails, the rejected assignment is blocked, newly observed diagnostic traces are added, and the repair loop continues.

B. Bound Variation and Joint Trace Encoding

MPTA-Repair follows the bound-variation view used in clock-bound repair [8]. Let a repairable clock-bound value be a numerical bound β occurring in an atomic clock constraint $x \sim \beta$, where x is a clock and $\sim \in \{<, \leq, =, \geq, >\}$ is fixed by the original model. For each such bound, MPTA-Repair introduces a bound variation variable v. The repaired constraint is written as:

$x \sim (\beta + v)$.

The current repair pipeline changes only the numerical bound. The clock reference, comparison operator, transition structure, location structure, synchronization labels, and reset set are not repair variables in the main experiment. Equality constraints can be represented in the same syntactic form when they occur in guards or invariants, but the operator itself is not changed.

For a diagnostic trace τ, MPTA-Repair constructs a TARTAR-style trace constraint system [8]. Let $X\tau$ denote the trace-local variables, including delay variables and clock values at trace positions, and let V denote the bound variation variables. The bound-varied trace constraint system is written as:

$T\tau\hat{}bv(X\tau, V)$.

It contains the usual constraints for clock initialization, time elapse, clock resets, guard satisfaction, and invariant satisfaction along the trace, except that bounds in guards and invariants are replaced by their varied forms [8]. Bound variation variables affect only the guard and invariant constraints in which the corresponding bound occurs.

For a violated property φ, the local repair obligation for a trace τ is:

$$Obl(\tau, \varphi, V) = (\exists X\tau \,.\, T\tau\hat{}bv(X\tau, V)) \wedge (\forall X\tau \,.\, T\tau\hat{}bv(X\tau, V) \Rightarrow \Phi\tau,\varphi(X\tau)).$$

The first conjunct requires that the same discrete trace skeleton remains executable after repair. This does not require the same delays or clock valuations as in the original counterexample; it only requires that the same sequence of discrete transitions still has at least one feasible timing realization. The second conjunct requires that no feasible timing realization of this trace skeleton still violates the considered property. Thus,

the local repair obligation does not block the functional path itself. It blocks the violating timing realizations of that path.

For a counterexample pool C, where each element is a property-trace pair (τ, φ), MPTA-Repair builds a joint hard constraint:

$$FH = \bigwedge_{(\tau,\varphi)\in C} Obl(\tau, \varphi, V) \wedge WF(V) \wedge Blocked(V).$$

Here, $WF(V)$ contains well-formedness constraints such as non-negative repaired bounds and consistency between lower and upper timing limits when both are present. $Blocked(V)$ excludes repair assignments that have already failed full-property regression or admissibility. To prefer small repairs, the MaxSMT soft constraints are:

$$FS = \bigwedge_{v\in V} (v = 0).$$

Maximizing the satisfied soft constraints minimizes the number of changed clock bounds [22]. When needed, additional optimization objectives minimize the total magnitude of the nonzero variations. This encoding generalizes single-trace clock-bound repair from one TDT to a pool of currently relevant property-trace obligations.

If a candidate assignment ρ is rejected, MPTA-Repair adds a blocking constraint that prevents the same assignment from being returned again. For the active variation variables $V\rho$ considered in that candidate, the exact blocking constraint can be written as:

$$Block\rho(V) = \bigvee_{v\in V\rho} v \neq \rho(v).$$

This does not claim that all similar assignments are invalid. It only prevents the refinement loop from repeating the same failed candidate.

C. Relation-Guided Search Optimization

As refinement proceeds, the number of violated properties, diagnostic traces, and editable clock bounds may grow [25]. Encoding every possible obligation and every editable bound without organization can make the MaxSMT problem larger and may lead to repeated solving over redundant evidence. MPTA-Repair therefore uses relation analysis as a search optimization. These relations guide pruning, grouping, and ranking, but every accepted repair is still validated by full-property regression and admissibility.

At the property layer, MPTA-Repair handles safety properties in a practical normalized fragment. A property is normalized into an A[]-style condition over location predicates and clock or integer comparisons [9]. The relation analysis currently uses conservative syntactic and formula-level rules. Two properties are treated as equivalent when their normalized forms are identical. A dominance relation is used only when it can be established safely. For example, for two upper-bound properties that are identical except for the numerical constant,

$A[]\ (Target \Rightarrow x \leq b1)$ and $A[]\ (Target \Rightarrow x \leq b2)$,

the first property dominates the second if $b1 \leq b2$. For lower-bound properties, the direction is reversed. Similar rules apply only when the target predicate, clock, and operator pattern match after normalization. Shared clocks, shared target locations, and shared templates are recorded as dependency information, but they are not by themselves used to remove obligations.

At the counterexample layer, MPTA-Repair maintains a pool of TDTs associated with violated properties. Exact duplicates are removed, and traces are annotated with their involved transitions, locations, clocks, guard bounds, invariant bounds, and source properties [8]. Structural similarity, shared prefixes, and shared repair variables are used to organize the pool and explain why certain traces should be considered together. However, a trace is removed from the active obligation set only when it is an exact duplicate or when a sound subsumption check shows that its local obligation is covered by another encoded obligation. Otherwise, the trace remains available for joint encoding.

At the repair-bound layer, MPTA-Repair maps each property-trace obligation to the clock-bound values that can affect it. This mapping helps restrict the active repair variables to those appearing in or influencing the current counterexample pool. It also helps order candidate generation when several bounds are available. However, relation scores do not prove correctness, and they do not force the solver to modify a particular bound. They only reduce the search effort by avoiding unrelated repair variables when the current counterexample evidence gives no reason to open them.

D. Optimization Objective

The MaxSMT objective is intentionally simple [27]. The primary objective is to minimize the number of changed clock bounds by maximizing soft constraints of the form $v = 0$. Among candidates with the same number of changed bounds, the implementation may minimize the total magnitude of the changes. This follows the engineering goal of producing small and inspectable repairs.

MPTA-Repair does not optimize directly for speculative notions such as semantic risk or causal relevance. Relation analysis is used before and around solving to select or rank repair variables and obligations. The solver objective itself remains centered on minimality of the edit set and, when enabled, magnitude of the edit values.

Static consistency constraints are enforced during candidate generation [28]. Repaired bounds must remain in the supported numerical domain. When a lower and an upper bound jointly constrain the same clock in a local region, the repaired values must not make the timing interval inconsistent. Since operators and clock references are fixed in the current repair space, generated assignments change only numerical bound values [12].

E. Validation, Admissibility, and Refinement

Candidate validation is the main correctness boundary of MPTA-Repair [29]. After applying a candidate edit set, the repaired model is verified against every property in Φ. A candidate that repairs the triggering property but violates another property is rejected. Such a failure means that the current encoded trace pool was insufficient: the candidate satisfied all encoded local obligations, but the repaired model

still has an execution that violates the complete specification. MPTA-Repair therefore collects the newly observed diagnostic trace and adds it to the pool for the next refinement round.

After full-property regression succeeds, MPTA-Repair applies the TARTAR-style admissibility test based on functional equivalence [8]. This check is needed because a clock-bound edit may satisfy safety properties by preventing intended behavior from occurring. TARTAR observes that timed-language equivalence is not a suitable admissibility criterion, because a successful timing repair is expected to remove at least some violating timed realizations [8]. Instead, functional equivalence compares the untimed languages of the original and repaired models [30]. Intuitively, the repaired model should not add or remove functional action traces, even though the timing of those traces may change.

In implementation, this criterion is checked by constructing untimed automata from the original and repaired timed models and comparing their accepted untimed languages [31]. If the languages differ, the repair is rejected and a separating behavior can be reported when available. Only candidates that pass both full-property regression and functional-equivalence admissibility are accepted.

The refinement loop therefore combines local repair generation with global validation. Local TDT encodings generate candidate bound assignments; full-property regression rejects candidates that do not satisfy the complete specification; admissibility rejects candidates that satisfy the properties by changing functional behavior; and blocking constraints prevent repeated failed assignments. The final result is a repair that satisfies the currently stated multi-property specification while preserving the intended untimed behavior of the model.

IV. Evaluation

A. Prototype Implementation

We implemented MPTA-Repair as a prototype repair pipeline around a timed-automata model parser, a verification interface, a diagnostic-trace analyzer, a MaxSMT-based repair backend, and an admissibility checker. The prototype uses UPPAAL verification to evaluate properties and collect timed diagnostic traces [19], and it uses Z3 to solve the repair constraints [28]. The implementation produces a structured repair report for each run, including violated properties, collected traces, selected repair variables, candidate edits, solver status, regression results, admissibility results, runtime, and failure reasons.

The repair frontend extracts repairable clock-bound occurrences from transition guards and location invariants. Each repairable site records the template, location or transition, clock, operator, original bound, and syntactic context. The trace analyzer maps diagnostic traces to the repair variables that occur along the corresponding execution. The relation analyzer builds property, trace, and repair-variable relations used for candidate ranking and decomposition. The model writer applies candidate bound edits and produces a repaired model for validation.

The admissibility module follows the TARTAR-style idea of functional equivalence [8]. It compares the untimed behavior of the original and repaired models and rejects candidates that add or remove functional behavior. This check is separate from full-property verification. A candidate must pass both the complete property set and admissibility to be considered successful.

B. Baseline

We compare MPTA-Repair with an iterative TARTAR-style baseline [10]. The baseline is intentionally stronger than a one-shot call to a single-trace repair tool. It verifies the current model, selects a violated property, collects a diagnostic trace, invokes a single-property clock-bound repair procedure, and validates each candidate against the same full-property regression and admissibility gates used by MPTA-Repair. If a candidate fails, the baseline continues with additional candidates or iterations until it finds a valid repair or reaches the same time and search budgets.

This baseline is fair because it gives single-property repair access to global validation. However, it still repairs internally from one property or one diagnostic trace at a time. It does not jointly analyze relations among multiple violated properties, multiple counterexamples, and repair variables before generating candidates.

C. Datasets

We evaluate the methods on two dataset groups. The first group consists of benchmark timed-automata models collected from UPPAAL and XTA benchmark sources. These models provide broader structural diversity and allow comparison with existing clock-bound repair settings. They are useful for testing whether the method remains effective on established benchmark examples rather than only on one family of case studies.

The second group is an industry dataset collected from practical industrial case studies. It contains timed-automata models related to railway alarm and communication scenarios [1], [15], train-gate behavior [16], automotive gear control [2], cardiac pacing [3], autonomous road-junction rules [17], mechanical ventilation [18], and aerospace scheduling [4]. These models are aligned with the industry-track motivation because they contain multi-component networks, synchronization among components, domain-level timing requirements, and properties that are not merely direct copies of individual guards or invariants.

For each reference model, we select time-safety properties that the original model satisfies. Properties are derived from the model documentation, source papers, embedded queries, and intended industrial requirements [9]. We filter out trivial properties that only mirror a single guard or invariant, and we distinguish repair specifications from auxiliary sanity checks or reachability queries.

D. Fault Injection

Faulty instances are generated by modifying clock-bound constants in guards and invariants. For simple mutants, one

bound is changed and the resulting model is retained if it violates at least one selected property. For complex mutants, two or more bounds are modified. We apply an effectiveness filter to avoid redundant multi-fault cases: a complex mutant is retained only if the injected edits are both meaningful, either because each single edit can independently cause a violation or because the combined edits create a violation that neither single edit causes alone.

The mutation process follows the standard motivation of mutation-based evaluation: real faulty timed-automata models are rarely available in sufficient quantity, so faults are seeded into verified reference models [26]. Since this paper studies clock-bound repair, the injected faults are restricted to the same repair space used by the methods [8]. Mutants that are syntactically invalid or outside the supported model fragment are discarded.

E. Research Questions and Metrics

The evaluation is organized around five research questions.

RQ1: Repair effectiveness. How often can each method find a repair that satisfies the full property set and passes admissibility?

RQ2: Multi-property benefit. How often does a local repair fix the triggering property but fail another property, and does full-property repair avoid these regressions?

RQ3: Multi-counterexample benefit. Does accumulating diagnostic traces improve repair robustness compared with repairing from a single trace?

RQ4: Relation-guided optimization. Do property, counterexample, and repair-variable relations improve repair success, runtime, or repair size?

RQ5: Failure analysis. Why do some cases remain unrepaired?

We report the number of models, properties, mutants, repaired cases, success rate, runtime, refinement rounds, counterexamples used, candidate repairs generated, changed bounds, edit magnitude, regression failures, admissibility rejections, timeouts, and unsupported cases. For industry cases, we additionally report model complexity metrics such as the number of templates, locations, transitions, clocks, and synchronization channels.

F. Overall Repair Effectiveness

Table I summarizes the overall repair effectiveness. On the benchmark dataset, the iterative TARTAR-style baseline repairs 72% of the cases, while MPTA-Repair repairs 93%. On the industry dataset, the difference is larger: the baseline repairs 20% of the cases, while MPTA-Repair repairs 76%. These results suggest that the proposed workflow is effective not only on general timed-automata benchmarks, but also on harder industrial models where several properties and traces may interact.

The baseline remains competitive on simpler benchmark cases because many faults are local and close to the original single-trace repair assumption. In these cases, one diagnostic trace often contains enough information to identify a useful clock-bound edit. MPTA-Repair improves on this setting mainly through candidate blocking, full-property regression, and counterexample accumulation.

TABLE I
OVERALL REPAIR OUTCOMES REPORTED IN THE SOURCE DRAFT.

Dataset group	Method	Cases	Repaired	Success rate
Benchmark	Iterative TARTAR-style baseline	N1	R1	72%
Benchmark	MPTA-Repair	N1	R2	93%
Industry	Iterative TARTAR-style baseline	N2	R3	20%
Industry	MPTA-Repair	N2	R4	76%

The industry results show a different pattern. Industrial case-study models contain interacting components, synchronization, observer templates, and domain-level properties. A repair that is locally valid for one trace is frequently rejected by full-property regression or admissibility. MPTA-Repair is more robust in this setting because it accumulates evidence from multiple properties and traces before committing to a repair.

G. Benefit of Full-Property Regression

Full-property regression rejects candidates that repair one property while breaking another. This failure mode appears frequently in multi-property models. For example, tightening an upper-bound guard may eliminate a late-response trace, but it may also prevent a required transition from occurring in another operating phase. Similarly, relaxing a timeout bound may satisfy a liveness-like response monitor while violating a safety monitor that constrains maximum waiting time.

The evaluation confirms that single-property success is not sufficient evidence of repair validity. Several baseline candidates satisfy the triggering property but fail another property in the same property file. MPTA-Repair avoids accepting such candidates because every candidate is checked against the conjunction of all selected properties. This supports the central design decision of treating property restoration as a global objective rather than as a sequence of independent local obligations.

H. Benefit of Multi-Counterexample Repair

Counterexample accumulation improves robustness by reducing overfitting to one diagnostic trace. In several cases, the first repair candidate removed the reported TDT but left another trace feasible. After the new trace was added to the counterexample pool, MPTA-Repair found a different bound edit or a small joint edit set that addressed both traces. This behavior is especially important in models with several operational modes, where different counterexamples expose different timing regions of the same underlying defect.

The number of refinement rounds remained modest in most successful cases. Successful repairs usually required one to three rounds, while harder cases accumulated more traces before either finding a repair or timing out. The main cost

of additional counterexamples is larger SMT and regression-verification effort, but the benefit is fewer locally overfitted repairs.

I. Relation-Guided Optimization

Relation-guided search improves both candidate quality and repair efficiency. Property relations help avoid repeatedly solving essentially equivalent obligations. Counterexample relations identify traces that share path fragments or repair variables. Repair-variable relations focus the solver on bounds that are close to the observed violations, rather than on all syntactically editable bounds in the model.

The ablation study shows that removing relation guidance increases the number of candidates explored and leads to more regression failures. Removing counterexample accumulation has a larger impact on industry cases than on benchmark cases, because industrial models more often produce multiple related violations. Removing admissibility increases apparent property success but accepts repairs that remove or add functional behavior, which are not valid under our repair criterion.

J. Repair Size and Runtime

Most successful repairs are small. The median repair changes one or two clock bounds, and the average edit magnitude remains close to the injected fault size. This is desirable for engineering use: a repair suggestion that changes a small number of meaningful timing thresholds is easier to inspect than a large set of unrelated edits.

Runtime is mainly affected by the number of active traces, the number of clocks appearing along those traces, and the number of repair variables opened for solving. This is consistent with prior observations in TDT-based repair: longer traces and larger constraint systems increase quantifier elimination and solving cost [8]. Industry models tend to be harder because they contain more templates, synchronizations, and property interactions, which increase both verification cost and admissibility-checking cost.

K. Failure Analysis

Not all cases are repairable within the current scope. We classify failures into five categories. First, some instances have no feasible candidate within the clock-bound repair space; the true repair may require changing a reset, synchronization label, data update, or model structure. Second, some candidates satisfy encoded traces but fail full-property regression. Third, some candidates satisfy all properties but fail admissibility because they remove or add untimed behavior. Fourth, some instances time out during solving, verification, or admissibility checking. Fifth, some models use unsupported syntax or property forms that fall outside the current implementation.

This failure analysis is useful for interpreting the results. MPTA-Repair is not a general timed-automata repair system; it is a clock-bound repair workflow. Failures outside this repair space should not be interpreted as failures of the central idea, but as evidence that richer repair actions and stronger trace generation will be needed for broader deployment.

V. Discussion and Threats to Validity

A. Lessons Learned

Lesson 1: Full-property regression is necessary. In industrial models, properties are not independent. A local fix can easily break a related requirement, so every candidate repair must be checked against the complete property set.

Lesson 2: Admissibility changes the meaning of repair success. Without behavior preservation, some repairs appear successful because they remove problematic behavior. For industrial systems, such repairs are not acceptable.

Lesson 3: One counterexample is often insufficient. A single TDT gives useful failure evidence, but it may expose only one timing realization. Accumulating traces helps avoid repairs that overfit the first observed violation.

Lesson 4: Relation analysis should be conservative. Property and trace relations are useful for search guidance, but uncertain relations should not be used to discard obligations. Final acceptance must remain based on verification and admissibility.

Lesson 5: Repair reports matter. For engineering use, a repair tool should explain not only the final edit, but also the violated properties, traces considered, candidate failures, regression results, and admissibility outcome.

B. Threats to Validity

Internal validity. The prototype includes parsers for timed-automata models, property files, diagnostic traces, repair sites, and admissibility artifacts. Bugs in parsing, trace reconstruction, SMT encoding, candidate application, or admissibility checking may affect the results. We mitigate this threat by validating every accepted candidate using the external model checker and by recording repair reports for each run.

Construct validity. The evaluation uses injected clock-bound faults. Such faults are appropriate for evaluating clock-bound repair, but they do not cover all real modeling errors. Real faults may involve wrong resets, missing transitions, incorrect synchronizations, data updates, or erroneous properties. The results should therefore be interpreted as evidence for clock-bound repair, not for arbitrary timed-automata repair.

External validity. Although the industry dataset covers several practical domains, it is still a finite set of open or reconstructed models. It may not represent all industrial timed-automata models. Larger proprietary models, richer data variables, and more complex property specifications may introduce additional challenges.

Baseline fairness. The baseline is an iterative TARTAR-style repair workflow adapted to the multi-property setting by adding full-property regression and admissibility checking. This makes it stronger than a one-shot baseline, but it is still not identical to the original setting for which TARTAR was designed. We therefore interpret the comparison as a comparison against a strong single-property repair strategy, not as a claim that TARTAR itself is unsuitable in general.

Property selection. Industry-case properties were selected from model descriptions, papers, embedded queries, and in-

tended timing requirements. Some properties may be incomplete or may not capture all domain-level requirements. We mitigate this by requiring all reference models to satisfy the selected property set and by filtering out trivial guard-mirror properties when constructing the benchmark.

Repair-space limitation. The current method focuses on numerical clock-bound edits in guards and invariants [8]. Some failed cases likely require repair actions outside this space. Extending the method to reset repair, synchronization repair, clock-reference repair, and structural repair is left for future work.

VI. Related Work

The closest related work is clock-bound repair for timed systems and TarTar [10]. Clock-bound repair encodes timed diagnostic traces into linear real arithmetic and uses MaxSMT solving to compute small modifications to guard and invariant bounds [8]. TarTar extends this line into a repair analysis tool and introduces an admissibility check based on functional equivalence. MPTA-Repair builds on this foundation, but targets a different setting: multiple properties, multiple diagnostic traces, and full-property validation in timed-automata models from practical industrial cases.

A second related direction is parameter synthesis for parametric timed automata [11]. Tools such as IMITATOR synthesize parameter valuations that satisfy timing constraints or preserve behavior around a reference valuation [32]. These techniques are powerful for design-space exploration and robustness analysis [13]. MPTA-Repair differs in its use of diagnostic traces and its focus on producing small syntactic repairs for faulty models rather than synthesizing a full parameter region.

A third related direction is model repair and automated program repair using SAT, MaxSAT, SMT, and counterexample-guided refinement [33]. These works provide search and optimization ideas such as fault localization [14], candidate blocking [34], repair minimization [35], and abstraction refinement [25]. MPTA-Repair adapts this style of reasoning to the semantics of timed automata, where candidate validity must consider both timing properties and untimed behavior preservation.

Finally, this work is related to industrial verification and dependability engineering [23]. Industry-oriented verification often emphasizes full regression, explainability, traceability, and lessons learned rather than only theoretical completeness [24]. MPTA-Repair follows this perspective by treating repair as an engineering workflow: diagnostic traces generate candidates, but acceptance depends on full regression evidence and admissibility.

VII. Conclusion

This paper presented **MPTA-Repair**, an automatic repair workflow for multi-property, multi-counterexample clock-bound repair of industrial timed-automata models. The method addresses a practical limitation of local TDT-based repair: industrial models are commonly validated against multiple timing properties, and one timing defect may produce several related counterexamples. Repairing one trace in isolation may therefore be insufficient.

MPTA-Repair collects diagnostic traces from violated properties, uses lightweight relations among properties, traces, and repairable bounds to guide repair search, and synthesizes small clock-bound edits using a MaxSMT-based backend. Every candidate is validated by full-property regression and a TARTAR-style admissibility check based on functional equivalence [8]. This design ensures that accepted repairs are not merely local counterexample fixes, but system-level timing corrections that preserve intended behavior.

Our evaluation on benchmark and industry timed-automata datasets shows that MPTA-Repair improves repair success over an iterative TARTAR-style baseline, especially on industry cases where multi-property interaction and admissibility constraints are prominent. The results also show that failures remain meaningful: unrepaired cases often require changes outside the clock-bound repair space or exceed the configured solving and validation budgets.

Future work will extend the repair space beyond numerical clock bounds to include resets, clock references, synchronization labels, and structural edits. We also plan to strengthen counterexample generation, improve relation-guided decomposition, and evaluate the method on larger industrial timed-automata models.

References

[1] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer," in *Proceedings of Formal Methods for Industrial Critical Systems*, 2021, pp. 176–193.

[2] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 1998, pp. 281–297.

[3] Z. Jiang, M. Pajic, R. Alur, and R. Mangharam, "Modeling and verification of a dual chamber implantable pacemaker," in *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2012, pp. 188–203.

[4] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "Herschel-planck software schedulability analysis using UPPAAL," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, 2010, pp. 282–296.

[5] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[6] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[7] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[8] M. Koelbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," 2019, manuscript.

[9] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[10] M. Koelbl, S. Leue, and T. Wies, "TarTar: A timed automata repair tool," 2020, arXiv:2002.02760.

[11] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the ACM Symposium on Theory of Computing*, 1993, pp. 592–601.

[12] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, 2019.

[13] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," 2021, arXiv:2108.08018.

[14] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, 2011, pp. 504–509.

[15] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," 2024, arXiv:2409.18732.

[16] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[17] A. Belguidoum, A. S. Al-Sibahi *et al.*, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Electronics*, vol. 10, no. 23, p. 3040, 2021.

[18] J. Cuartas *et al.*, "Formal verification of a mechanical ventilator using UPPAAL," in *Proceedings of Formal Techniques for Safety-Critical Systems*, 2023.

[19] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[20] P. Han, Z. Zhai, B. Nielsen, and U. Nyman, "A modeling framework for schedulability analysis of distributed avionics systems," 2018, arXiv:1803.11050.

[21] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[22] R. Sebastiani and S. Tomasi, "Optimization modulo theories with linear rational costs," *ACM Transactions on Computational Logic*, vol. 16, no. 2, 2015.

[23] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.

[24] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Springer, 2018.

[25] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proceedings of Computer Aided Verification*, 2000, pp. 154–169.

[26] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[27] R. Sebastiani and P. Trentin, "On optimization modulo theories, MaxSMT and sorting networks," 2017, arXiv:1702.02385.

[28] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.

[29] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[30] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.

[31] P. Bouyer, P. Gastin, F. Herbreteau, O. Sankur, and B. Srivathsan, "Zone-based verification of timed automata: Extrapolations, simulations and what next?" 2022, arXiv:2207.07479.

[32] Étienne André, "IMITATOR II: A tool for solving the good parameters problem in timed automata," 2010, arXiv:1011.0223.

[33] W. Weimer, T. Nguyen, C. L. Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the International Conference on Software Engineering*, 2009, pp. 364–374.

[34] S. Mechtaev, J. Yi, and A. Roychoudhury, "Angelix: Scalable multiline program patch synthesis via symbolic analysis," in *Proceedings of the International Conference on Software Engineering*, 2016, pp. 691–701.

[35] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, "SemFix: Program repair via semantic analysis," in *Proceedings of the International Conference on Software Engineering*, 2013, pp. 772–781.